

Open in app ↗

Sign up

Sign in

Medium

 Search

Android

+

Java

+

Nfc

+

Ntag 424

+

Sdm

+

Demystify the Secure Dynamic Message with NTAG 424 DNA NFC tags (Android/Java) Part 2

19 min read · Jun 19, 2024



AndroidCrypto

Follow



Listen



Share

This article will help you in understanding the “mysterious” **Secure Dynamic Message** (“SDM”) or **Secure Unique Number** (“SUN”) feature, available in NXP’s NTAG 424 DNA NFC tags.

This is **part 2** of the article series about the **Secure Dynamic Message** (“SDM”) feature. It is the practical implementation in an Android app, in **part 1** you get a lot of information about the complete data management and you should read part 1 before working with part 2.

Update October 12th 2025

As I received a lot of requests, I added an article that gives in detail “[A comprehensive overview of all keys for the NTAG424 NFC chip](#)”.

Just some notes at the beginning: as the sample app accompanies a tutorial the code is not optimized in any kind. Second: I setup a separate activity for each topic to show how the specific topic is reachable — of course you could use a “one activity” for all topics. Third: As the author of the “[ntag424-java](#)” library, **Jonathan Bartlett**, is considering the library “still in development” I decided to include the library as code (and not as dependency) to avoid any unwanted effects due to further enhancements.

These topics are covered by the sample app:

- **General note** on working with NTAG424DNA tags
- **Prepare the tag** by uploading a Capability Container and prepare the key settings
- write a NDEF message with **Plaintext Tag data**
- write a NDEF message with **Encrypted Tag data** (“PICC data”)
- write a NDEF message with **Encrypted Tag data** (“PICC data”) and **Encrypted File data**
- write a NDEF message with Plaintext Tag data and activated **Read Counter Limit**
- write a NDEF message with Encrypted Tag data (“PICC data”) and Encrypted File data using **static custom keys**
- write a NDEF message with Encrypted Tag data (“PICC data”) and Encrypted File data using **diversified custom keys**
- **NDEF Reader**: read the NDEF message, (if encrypted, decrypted the data) and **validate the CMAC**
- **Tag Overview** - gives an overview about the most important data on the tag (authentication scheme, application key values and file data)
- **Unset the tag** to default settings

In the next steps we are going to activate an option and then check the content, for that reason you need to call your task manager on the smartphone and go back to your “Home screen” or move back to the main activity and choose the “NDEF Reader” menu.

General note on reading, writing and changing data on NTAG424DNA tags

This modern tag is using a security scheme that is based on AES keys and encryption. For mostly all operation you need to **authenticate** with the tag before you are allowed to run the activity, proving that you know the keys stored on the tag. To improve the security there is a second authentication scheme available with these tags named “*Leakage Resilient Primitive, or LRP, a wrapper around the AES cryptography to enhance side-channel attack resistance*”. The LRP can get activated just

one time, and when it is set there is no way back to the AES authentication.

Unfortunately there is no direct way to know what authentication scheme is active, the only way is to run an authentication in one mode (I'm starting with AES in my sample app), and depending on the result I'm trying to run the LRP in case of a failure. To set some features later in the workflow you need to know if this tag is running in AES or LRP mode.

This is the authentication workflow I'm using in nearly all activities to authenticate and detect the authentication scheme:

```
boolean isLrpAuthenticationMode = false;

success = AESEncryptionMode.authenticateEV2(dnaC, ACCESS_KEY0, Ntag424.FACTORY_
if (success) {
    writeToUiAppend(output, "AES Authentication SUCCESS");
} else {
    // if the returnCode is '919d' = permission denied the tag is in LRP mode a
    if (dnaC.getLastCommandResult().status2 == PERMISSION_DENIED) {
        // try to run the LRP authentication
        success = LRPEncryptionMode.authenticateLRP(dnaC, ACCESS_KEY0, Ntag424.
        if (success) {
            writeToUiAppend(output, "LRP Authentication SUCCESS");
            isLrpAuthenticationMode = true;
        } else {
            writeToUiAppend(output, "LRP Authentication FAILURE");
            writeToUiAppend(output, "returnCode is " + Utils.byteToHex(dnaC.get
            writeToUiAppend(output, "Authentication not possible, Operation abc
            return;
        }
    } else {
        // any other error, print the error code and return
        writeToUiAppend(output, "AES Authentication FAILURE");
        writeToUiAppend(output, "returnCode is " + Utils.byteToHex(dnaC.getLast
        return;
    }
}
```

Download and install the NXP TagInfo app

I strongly recommend to download and install the TagInfo app from NXP (this company invented this tag); download it from the PlayStore here:

<https://play.google.com/store/apps/details?id=com.nxp.taginfolite&hl=gsw&gl=US>.

Using this app you get an overview about the status of your tag — this is an example output:

```
# Configuration Information:
Secure Dynamic Messaging: Enabled
UID Mirroring: Enabled
SDM Read Counter: Enabled
SDM Read Counter Limit: Enabled
SDM Read Counter Limit Value: 0x030000
Encrypted File Data Mirroring: Disabled
...
Settings for file 02:
* FileType: StandardData file
* Secure Dynamic Messaging: enabled
* Communication mode: plain
* Permissions:
  - ReadWrite: with key 0x2
  - Change: with key 0x0
  - Read: free access
  - Write: with key 0x2
* File size: 256 bytes
* SDM mirror options:
  - UID mirror enabled
  - SDMReadCtr enabled
  - SDMReadCtrLimit enabled
  - ASCII encoding
* SDM Access rights:
  - SDMCtrRet permissions: no access
  - Meta Read: Plain PICCData mirror
  - File Read: with key 0x2
* UID Mirror offset: 0x2E0000
* SDMReadCtr mirror offset: 0x410000
* SDM MAC input offset: 0x4D0000
* SDM MAC mirror offset: 0x4D0000
* SDMReadCtr limit: 0x030000

[000] 00 5B D1 01 57 55 00 68 74 74 70 73 3A 2F 2F 73 |NUL[.sohWUNULhttps://s|
[010] 64 6D 2E 6E 66 63 64 65 76 65 6C 6F 70 65 72 2E |dm.nfcdeveloper.|
[020] 63 6F 6D 2F 74 61 67 70 74 3F 75 69 64 3D 2A 2A |com/tagpt?uid=**|
[030] 2A 2A 2A 2A 2A 2A 2A 2A 2A 2A 2A 2A 26 63 74 72 |*****&ctr|
[040] 3D 2A 2A 2A 2A 2A 2A 26 63 6D 61 63 3D 2A 2A 2A |=*****&cmac=***|
[050] 2A 2A 2A 2A 2A 2A 2A 2A 2A 2A 2A 2A 2A 00 00 00 |*****NULNULNUL|
```

Prepare the tag by uploading a Capability Container and prepare the key settings

You may ask — “why are you uploading (writing) a Capability Container (“CC”) as there is already one in place ?” It’s easy to answer: you may have played a little bit

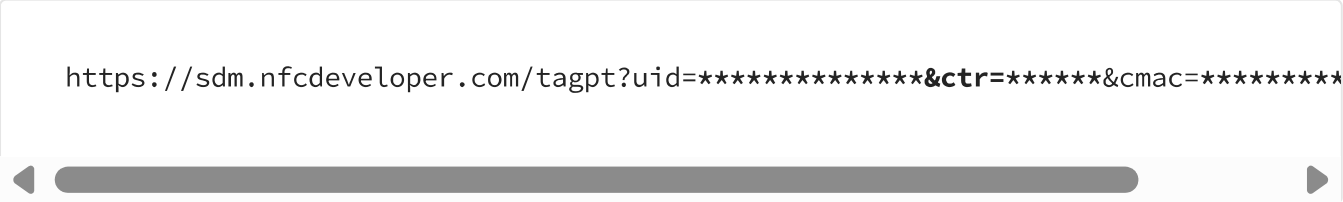
The next step is to write an NDEF template on the tag so that the tag will deliver this template when tapped to a reader — but without any tag specific data:

```
ndefRecord = master.generateNdefTemplateFromUrlString("https://sdm.nfcdeveloper
```

The last setting is to change the AES-128 key value of the Application Keys number 3 and 4. You can choose between static custom keys or diversified keys by selecting the radio button. In CUSTOM key mode these key values are set:

In DIVERSIFIED key mode the Application Key is like above, but App Key number 4 gets diversified (see the the chapter at the end for details).

Now the tag is well prepared for activating the SDM feature in the next activities. For a first test, simply tap the tag to your smartphone with no active app and see what happens. On my device I'm getting asked which browser should be used for the URL type NDEF message; after selecting "Chrome" in my case the browser will contact the server using this URL:



```
https://sdm.nfcdeveloper.com/tagpt?uid=*****&ctr=*****&cmac=*****
```

It's easy to see that the tag is prepared to deliver the tag UID, the Read Counter and calculates a CMAC over the data, but there is no real data, just a lot of placeholders waiting for activation. For that reason the backend server does not validate the provided data.

Write a NDEF message with Plaintext Tag data

Now we are activating the SDM feature by using the command "changeFileSettings". The "Setup Plaintext SUN Message" activity has three options to choose from:

- Plaintext data with tag UID only
- Plaintext data with Read Counter only
- Plaintext data with tag UID and Read Counter (default setting).

I'm explaining the settings for the last option only:

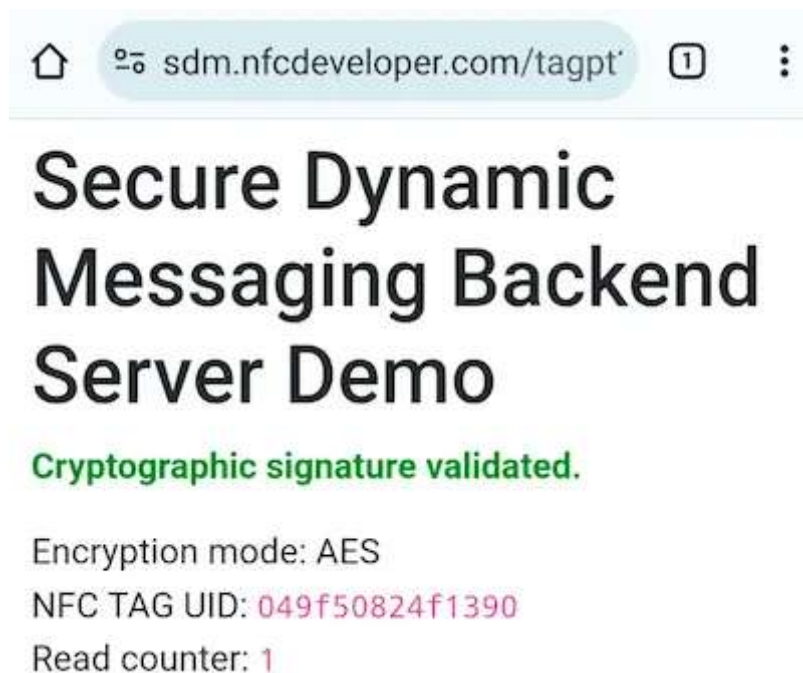
```
SDMSettings sdmSettings = new SDMSettings();
sdmSettings.sdmEnabled = true; // at this point we are just preparing the templ
sdmSettings.sdmMetaReadPerm = ACCESS_EVERYONE; // Set to a key to get encrypted
sdmSettings.sdmFileReadPerm = ACCESS_KEY2; // Used to create the MAC and En
sdmSettings.sdmReadCounterRetrievalPerm = ACCESS_NONE; // Not sure what this is
sdmSettings.sdmOptionEncryptFileData = false;
byte[] ndefRecord = null;
NdefTemplateMaster master = new NdefTemplateMaster();
master.usesLRP = isLrpAuthenticationMode;
master.fileDataLength = 0; // no (encrypted) file data
if (rbUid.isChecked()) {
    ndefRecord = master.generateNdefTemplateFromUrlString("https://sdm.nfcdevel
} else if (rbCounter.isChecked()) {
    ndefRecord = master.generateNdefTemplateFromUrlString("https://sdm.nfcdevel
```

```
} else {  
    ndefRecord = master.generateNdefTemplateFromUrlString("https://sdm.nfcdevel  
}
```

The SDM feature is enabled by “sdmEnabled = true”, followed by 2 keys that are set: the **sdmMetaReadPermission** is responsible for en- and decrypt (encrypted) PICC data. As our SDM message does not include any encrypted data the key setting is “ACCESS_EVERYONE”.

The second key is “sdmFileReadPermission” — this key is responsible for calculating the CMAC (necessary) and encrypted file data (Note: we don’t have any file data in this activity, for that reason the fileDataLength is 0).

Please use the UID & Read Counter option (default) - when you tap your tag to the reader, wait for the activation and then go back to your Home screen. Tap the tag again and use your browser to validate the NDEF message:



This was the delivered NDEF message with this URL:

[https://sdm.nfcdeveloper.com/tagpt?
uid=049F50824F1390&ctr=000001&cmac=2446E527C37E073A](https://sdm.nfcdeveloper.com/tagpt?uid=049F50824F1390&ctr=000001&cmac=2446E527C37E073A)

Congratulations — you created your first Secure Dynamic Message !

Now we are choosing the first option — the “tag UID only” — and tap the tag again to the reader. After activation, moving away from this activity (“Home Screen”) and tapping again (this is the produced URL: <https://sdm.nfcdeveloper.com/tagpt?uid=049F50824F1390&cmac=B1EE1FD5DC0D9654>):



Analyzing the URL you see that the tag UID together with the CMAC is populated, but why did we receive a “Bad Request” error ? The reason is easy, this backend server does validate URLs with UID and Read Counter data only. For this reason I included the “NDEF Reader Activity” that acts like a backend server.

NDEF Reader: read the NDEF message and validate the CMAC

Start the NDEF Reader activity and tap the tag. As we are using Plaintext data just leave the first (default) option:



This is the validating result:



As we already know there is just the UID and the CMAC is validated. We will return to this activity in the next steps, but now you should be able to use the last Plaintext option (Read Counter only) and validation on your own.

Write a NDEF message with Encrypted Tag data (“PICC data”)

Starting with this step we are leaving the “human readable” NDEF message type and switching to a more secure way. In the tag documentation the tag is always named “PICC”, so we are using this naming here as well. Start the “**Setup Encrypted SUN Message**” activity uses the same options like the plaintext one; we are starting with the default (UID and Read Counter) option:

NTAG 424 DNA SDM Feature

Encrypted SUN Feature

Write the URL template for an Encrypted SUN Message to file 2 on the tag

Select the Output Fields

☐ (PICC with UID only) + CMAC (endpoint is not working, use NDEF Reader instead)

☐ (PICC with Counter only) + CMAC (endpoint is not working, use NDEF Reader instead)

☒ (PICC with UID + Counter) + CMAC

Please note: when a NTAG424DNA tag is tapped to the NFC reader the action runs without any further confirmation.

operation output

```
== FINISHED ==  
File 02h Change File Settings  
SUCCESS  
File 02h Writing the NDEF URL  
Template SUCCESS  
getFileSettings File 02 AUTH-KEY  
RW Is: 2  
AES Authentication SUCCESS  
NFC tag connected  
Tag ID: 049f50824f1390
```

This is the way how the tag is setup for encrypted PICC data:

```

SDMSettings sdmSettings = new SDMSettings();
sdmSettings.sdmEnabled = true; // at this point we are just preparing the templ
sdmSettings.sdmMetaReadPerm = ACCESS_KEY2; // Set to a key to get encrypted PIC
sdmSettings.sdmFileReadPerm = ACCESS_KEY2; // Used to create the MAC and Encryp
sdmSettings.sdmReadCounterRetrievalPerm = ACCESS_NONE; // Not sure what this is
sdmSettings.sdmOptionEncryptFileData = false;
byte[] ndefRecord = null;
NdefTemplateMaster master = new NdefTemplateMaster();
master.usesLRP = isLrpAuthenticationMode;
master.fileDataLength = 0; // no (encrypted) file data
if (rbUid.isChecked()) {
    sdmSettings.sdmOptionUid = true;
    sdmSettings.sdmOptionReadCounter = false;
} else if (rbCounter.isChecked()) {
    sdmSettings.sdmOptionUid = false;
    sdmSettings.sdmOptionReadCounter = true;
} else {
    sdmSettings.sdmOptionUid = true;
    sdmSettings.sdmOptionReadCounter = true;
}
ndefRecord = master.generateNdefTemplateFromUrlString("https://sdm.nfcdeveloper

```

It is important to define the “**sdmMetaReadPermission**” key as this key is used to encrypt the PICC data. Please note: DO NOT diversify this key using the tag UID because during the decryption phase the tag UID is not known !

As we don not use Encrypted File data the “sdmOptionEncryptFileData” flag is false and the “fileDataLength” is 0.

Tap your tag to the reader, move to your “Home Screen”, tag again and run your browser (this is the URL in use: https://sdm.nfcdeveloper.com/tag?picc_data=98A6DBED29A51036B8B875ECD390ABED&cmac=EEAAE975B350A4A5).

Note: the result is different to the Plaintext one, the backend server is receiving “/tag?” instead of “/tagpt?”):

Secure Dynamic Messaging Backend Server Demo

Cryptographic signature validated.

Encryption mode: AES

PICC Data Tag: c7

NFC TAG UID: 049f50824f1390

Read counter: 5

As you can see — the Read Counter is now at number 5 as we tapped the tag multiple times to the reader. For validating you can use the NDEF Reader of course as well.

Get AndroidCrypto's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Now activate and validate the other 2 options on your own (be aware to use the NDEF Reader).

Write a NDEF message with Encrypted Tag data ("PICC data") and Encrypted File data

This is the most sophisticated NDEF message this tag can deliver. Run the "Setup Encrypted File SUN Message" activity; as per data sheet only the tag UID and Read Counter option is available. Tap the tag to the reader, activate this and move to your "Home Screen". Tap the tag again, run you browser and see the results of this URL
https://sdm.nfcdeveloper.com/tag?picc_data=D570FA028BE069B3C3DFAA42AC361B2D&enc=EBB1044D5E036BD71681F3CE82BA9D8452D82DED14424A0EDFB25FC1425B98D9&cmac=6AB84D8EC134D904":

Secure Dynamic Messaging Backend Server Demo

Cryptographic signature validated.

Encryption mode: AES

PICC Data Tag: c7

NFC TAG UID: 049f50824f1390

Read counter: 7

File data (hex):

33312e30352e323032342031393a32323a3437233

13233342a2a2a2a2a2a2a

File data (UTF-8): 31.05.2024

19:22:47#1234*****

Some more data as before are populated now:

- the tag UID
- the Read Counter
- File Data in Hex encoding and in UTF-8 encoding (looks like a timestamp, followed by '#1234' and some placeholders)

This is the code I have used to get this NDEF message:

```
SDMSettings sdmSettings = new SDMSettings();
sdmSettings.sdmEnabled = true; // at this point we are just preparing the templ
sdmSettings.sdmMetaReadPerm = ACCESS_KEY2; // Set to a key to get encrypted PIC
sdmSettings.sdmFileReadPerm = ACCESS_KEY2; // Used to create the MAC and Encryp
sdmSettings.sdmReadCounterRetrievalPerm = ACCESS_NONE; // Not sure what this is
sdmSettings.sdmOptionEncryptFileData = true;
byte[] ndefRecord = null;
NdefTemplateMaster master = new NdefTemplateMaster();
master.usesLRP = isLrpAuthenticationMode;
master.fileDataLength = 32; // encrypted file data available. The timestamp is
if (rbUid.isChecked()) {
    sdmSettings.sdmOptionUid = true;
    sdmSettings.sdmOptionReadCounter = false;
```

```

} else if (rbCounter.isChecked()) {
    sdmSettings.sdmOptionUid = false;
    sdmSettings.sdmOptionReadCounter = true;
} else {
    sdmSettings.sdmOptionUid = true;
    sdmSettings.sdmOptionReadCounter = true;
}
ndefRecord = master.generateNdefTemplateFromUrlString("https://sdm.nfcdeveloper

```

This time the flag “sdmOptionEncryptFileData” is true and we set the “master.fileDataLength” to “32”, meaning our encrypted file data has a length of 32 bytes or characters. I wrote the encrypted file data during the “Setup the Encrypted File SUN Message” activity and it was the timestamp (“31.05.2024 19:22:47”) followed by a static string (“#1234”), in total 24 characters. As the Encrypted File data is 32 characters long, the library filled the missing 8 characters with “*****”.

Important: the **Encrypted File data length has to be a multiple of 16**, and to write the data to the file we need an offset for writing, as it is not written at the beginning. I tested the value by writing the template and counting the position, in my example it starts with an offset “87” (if the tag uses LRP encryption the offset is (“87 + 16” = “103”):

```

byte[] fileData = (getTimestampLog() + "#1234").getBytes(StandardCharsets.UTF_8);
try {
    if (isLrpAuthenticationMode) {
        WriteData.run(dnaC, NDEF_FILE_NUMBER, fileData, (87 + 16)); // LRP Encr
    } else {
        WriteData.run(dnaC, NDEF_FILE_NUMBER, fileData, 87);
    }
} catch (IOException e) {
    Log.e(TAG, "writeFileData IOException: " + e.getMessage());
    writeToUiAppend(output, "File 02h writeFileDataIOException: " + e.getMessage());
    writeToUiAppend(output, "Writing the File Data FAILURE, Operation aborted");
    return;
}

```

You may have counted the number of characters of the “enc” (= Encrypted File data) part:

```
EBB1044D5E036BD71681F3CE82BA9D8452D82DED14424A0EDFB25FC1425B98D9
```

This are 64 (hex) characters: because the “data to encrypt” is 32 bytes the data gets encrypted to 32 bytes of cipher text, but then encoded in Hex encoding, giving 64 characters.

Write a NDEF message with Plaintext Tag data and activated Read Counter Limit

This is a special security feature to minimize the chance of an attacker to gather more data samples to decrypt. You can use this feature in Plaintext or Encrypted PICC data options; I’m using the Plaintext one here as the effect is more easy to see. Run the “Setup Plaintext with Read Counter Limit” activity (just the full data option is setup here). After tapping the tag a third time the backend server is showing this data:

Secure Dynamic Messaging Backend Server Demo

Cryptographic signature validated.

Encryption mode: AES

NFC TAG UID: 049f50824f1390

Read counter: 3

But what does happen when we try to tap the tag a fourth time ? Try it out and wait for the result !

Using my Android phone the smartphone recognizes an NFC tag, but there seems to be no NDEF message populated by the tag. This is the expected behavior of the tag — no more (secret) data is given out. To reuse the tag you need to disable the feature and you need to run the “Unset the SUN feature” activity, “Prepare the tag” and run any setup (or in short: this is not a permanent “bricked” tag).

This is the code I was using to setup this feature (see the last 2 lines of code are important):

```
SDMSettings sdmSettings = new SDMSettings();
sdmSettings.sdmEnabled = true; // at this point we are just preparing the template
sdmSettings.sdmMetaReadPerm = ACCESS_EVERYONE; // Set to a key to get encrypted meta
sdmSettings.sdmFileReadPerm = ACCESS_KEY2; // Used to create the MAC and Encrypted File
sdmSettings.sdmReadCounterRetrievalPerm = ACCESS_NONE; // Not sure what this is for
sdmSettings.sdmOptionEncryptFileData = false;
sdmSettings.sdmOptionReadCounterLimit = true;
sdmSettings.sdmReadCounterLimit = 3; // here fixed to 3 readings
```

Write a NDEF message with Encrypted Tag data (“PICC data”) and Encrypted File data using static CUSTOM keys

This is a special version of the “Setup Encrypted File SUN message” activity as it does not use the default (“nulled”) AES keys but **static Custom Keys**:

```
byte[] APPLICATION_KEY_3 = Utils.hexStringToByteArray("A300000000000000000000000000000000");  
byte[] APPLICATION_KEY_4 = Utils.hexStringToByteArray("A400000000000000000000000000000000");
```

The key values were changed in the “Prepare the tag” activity – for details on how to change the keys use this code:

```
success = false;
try {
    ChangeKey.run(dnaC, ACCESS_KEY3, APPLICATION_KEY_DEFAULT , APPLICATION_KEY_
    success = true;
} catch (IOException e) {
    Log.e(TAG, "ChangeKey 3 IOException: " + e.getMessage());
}
```

Note: when using keys different from the default values the backend server (sdm.nfcdeveloper.com) is not been able to decrypt data and verify the signature,

please use the NDEF Reader activity instead.

Write a NDEF message with Encrypted Tag data (“PICC data”) and Encrypted File data using DIVERSIFIED CUSTOM keys

Using diversified (custom) keys instead of static keys is a security feature as well.

Due to the diversification nature the key changes for every (different) tag UID.

Again: do not diversify the key used for decrypting the (encrypted) PICC data (“SDM Meta Read Permission” key), as you usually don’t know the tag UID before decryption !

The NTAG424 library has a built-in feature for the diversification and you need to provide 2 “Master Keys”:

- MASTER_APPLICATION_KEY_FOR_DIVERSIFYING:

```
byte[] MASTER_APPLICATION_KEY_FOR_DIVERSIFYING = Utils.hexStringToByteArray("A5
```

- SYSTEM_IDENTIFIER_FOR_DIVERSIFYING

```
byte[] SYSTEM_IDENTIFIER_FOR_DIVERSIFYING = Utils.hexStringToByteArray("666F6F"
```

The third parameter for the diversification is the tag UID:

```
newKey4 = keyInfo.generateKeyForCardUid(realTagUid);
```

```
KeyInfo keyInfo = new KeyInfo();  
keyInfo.diversifyKeys = true;  
keyInfo.key = MASTER_APPLICATION_KEY_FOR_DIVERSIFYING.clone();  
keyInfo.systemIdentifier = SYSTEM_IDENTIFIER_FOR_DIVERSIFYING; // static value  
newKey4 = keyInfo.generateKeyForCardUid(realTagUid);
```

All other methods are the same as in “Write a NDEF message with Encrypted Tag data (“PICC data”) and Encrypted File data using static CUSTOM keys” activity.

Note: don’t forget to use the “NDEF Reader” activity for decryption and verification as we are using non default keys.

NDEF Reader activity



This activity is acting like a backend server — verify the calculated CMAC, decrypt PICC and File data. As the tag does not indicate what key (Default, Custom or Diversified) you need to select the “Key Type” before tapping a tag to the reader. If the NDEF message contains Plaintext data only the CMAC is verified:

```
operation output  
  
== FINISHED ==  
The CMAC is VALIDATED (AES)  
CMAC:B1EE1FD5DC0D9654  
Counter:  
UID:049F50824F1390  
-----  
NDEF message:  
https://sdm.nfcdeveloper.com  
/tagpt?uid=049F50824F1390&cmac=  
B1EE1FD5DC0D9654  
=====
```

If the NDEF message is containing encrypted PICC and File data the activity will decrypt the data and verify the CMAC:

```

operation output
== FINISHED ==
The CMAC is VALIDATED
cmacCalc length: 8 data:
3cb6a5b63b316835
Decrypted File data:
10.06.2024 15:26:00#1234*****
Encrypted File Data:
FC55F11ABB2502B0449EB0E1AF9121EF
0C07AC8D5499693D9BE7E63A1CCA2F4C
CMAC:3CB6A5B63B316835
Counter:2
UID:049f50824f1390
PICC:4698FC024FF3F463627D8D637E0
4E8C4
-----
NDEF message:
https://sdm.nfcdeveloper.com/tag
?picc_data=
4698FC024FF3F463627D8D637E04E8C4
&enc=
FC55F11ABB2502B0449EB0E1AF9121EF
0C07AC8D5499693D9BE7E63A1CCA2F4C
&cmac=3CB6A5B63B316835
=====
NFC tag connected
Tag ID: 049f50824f1390

```

The NDEF Reader differs between Plaintext and Encrypted Data by reading the URL (if the URL is starting with “tagpt?” the following data is Plaintext, if the string is “tag?” it is Encrypted data):

```

boolean isEncrypted = false;
if (fullPayload.startsWith("tagpt?")) {
    // plaintext data
    isEncrypted = false;
} else if (fullPayload.startsWith("tag?")) {
    isEncrypted = true;
} else {
    // not a matching api
    writeToUiAppend(output, "The Backend Server URL is not matching an API, abc
return;

```

```
}

```

The (encrypted) PICC data (Tag UID and Read Counter) is selected like this:

```
if (fullPayload.contains("picc_data=")) {
    isPiccData = true;
    // the picc data is 32 characters hex data (AES) or 48 characters (LRP)
    startIndex = fullPayload.indexOf("picc_data=") + 10;
    // in the first step I'm trying to read 48 characters and convert them to a
    // '&cmac' the conversion will fail, then I'm trying to use 32 characters i
    encryptedPiccData = fullPayload.substring(startIndex, startIndex + 48); //
    boolean isHexNumeric = Utils.isHexNumeric(encryptedPiccData);
    if (isHexNumeric == false) {
        // try with 32 characters for AES
        encryptedPiccData = fullPayload.substring(startIndex, startIndex + 32);
        isHexNumeric = Utils.isHexNumeric(encryptedPiccData);
        if (isHexNumeric == false) {
            Log.e(TAG, "Tried to find encrypted PICC data but none is matching
            writeToUiAppend(output, "Tried to find encrypted PICC data but none
            return;
        } else {
            Log.d(TAG, "Encrypted PICC Data found for AES");
            isLrpAuthentication = false;
        }
    } else {
        Log.d(TAG, "Encrypted PICC Data found for LRP");
        isLrpAuthentication = true;
    }
}
}
```

Depending on the encrypted data length the data is AES or LRP mode encrypted. In case of encrypted data the first step is to decrypt the PICC data:

```
byte[] encryptedPiccDataBytes;
if (isPiccData) {
    encryptedPiccDataBytes = Utils.hexStringToByteArray(encryptedPiccData);
} else {
    writeToUiAppend(output, "Could not find PICC data, aborted");
    return;
}
```

```
//System.out.println(Utils.printData("encryptedPiccDataBytes", encryptedPiccData);
piccData = PiccData.decodeFromEncryptedBytes(encryptedPiccDataBytes, encryptedF
byte[] uidDecrypted = piccData.getUid();
int readCounterDecrypted = piccData.getReadCounter();
```

If the key was diversified we need to diversify the key like follows:

```
// diversify the Master Application key with real Tag UID
KeyInfo keyInfo = new KeyInfo();
keyInfo.diversifyKeys = true;
keyInfo.key = MASTER_APPLICATION_KEY_FOR_DIVERSIFYING.clone();
keyInfo.systemIdentifier = SYSTEM_IDENTIFIER_FOR_DIVERSIFYING; // static value
cmacKey = keyInfo.generateKeyForCardUid(uidDecrypted);
```

Using this CMAC key we are now been able to decrypt the file data and verify the CMAC:

```
piccData.setMacFileKey(cmacKey);
byte[] cmacCalc = piccData.performShortCMAC(null); // null if MAC on PICC-only
byte[] cmacBytes;
if (isCmac) {
    cmacBytes = Utils.hexStringToByteArray(cmac);
} else {
    writeToUiAppend(output, "Could not find CMAC data, aborted");
    return;
}
```

Last but not least we are decrypting the file data and validating the CMAC using the **sdmFileReadPermission** key:

```
byte[] encryptedFileDataBytes = Utils.hexStringToByteArray(encryptedFileData);
byte[] decryptedFileData = piccData.decryptFileData(encryptedFileDataBytes);
writeToUiAppend(output, "Decrypted File data:\n" + new String(decryptedFileData);
// validate the CMAC over all elements
// The CMAC is calculated over the encrypted file data STRING (upper case hex c
```

```
// following '&cmac=' string. After concatenating both the string is converted
byte[] encryptedFileDataForCmac = (encryptedFileData + "&cmac=").getBytes(Stanc
cmacCalc = piccData.performShortCMAC(encryptedFileDataForCmac); // null if MAC
writeToUiAppend(output, printData("cmacCalc", cmacCalc));
isCmacValidated = Arrays.equals(cmacCalc, cmacBytes);
if (isCmacValidated) {
    writeToUiAppend(output, "The CMAC is VALIDATED");
} else {
    writeToUiAppend(output, "The CMAC is VOID");
}
```

Tag Overview — gives an overview about the most important data on the tag (authentication scheme, application key values and file data)

When playing around with the tags you will find out that it is hard to remember what you have changed on the tags. This activity tries to get the most important data by “try and error” with the tag. It will run an authentication with AES or LRP to find out the authentication scheme. It tries to authenticate using the application keys 1 to 4 and using the Default, Custom and Diversified Key values. Then the app reads the file settings of the 3 Standard Files 1 to 3, followed by reading the file data of the 3 files. Here is an example output:

```
=====
Authentication with FACTORY ACCESS_KEY 0
AES Authentication SUCCESS
-----
App Key 1 is FACTORY key
App Key 2 is FACTORY key
App Key 3 is CUSTOM key
App Key 4 is CUSTOM key
=====
Get the File Settings

= FileSettings =
fileNumber: 1
fileType: n/a
commMode: PLAIN
accessRights RW:      0
accessRights CAR:     0
accessRights R:       14
accessRights W:       0
fileSize: 32
= Secure Dynamic Messaging =
isSdmEnabled: false
isSdmOptionUid: false
isSdmOptionReadCounter: true
```

```

isSdmOptionReadCounterLimit: false
isSdmOptionEncryptFileData: false
isSdmOptionUseAscii: true
sdmMetaReadPerm:          14
sdmFileReadPerm:          14
sdmReadCounterRetrievalPerm: 14
-----
= FileSettings =
fileNumber: 2
fileType: n/a
commMode: PLAIN
accessRights RW:          2
accessRights CAR:         0
accessRights R:           14
accessRights W:           2
fileSize: 256
= Secure Dynamic Messaging =
isSdmEnabled: true
isSdmOptionUid: true
isSdmOptionReadCounter: true
isSdmOptionReadCounterLimit: false
isSdmOptionEncryptFileData: true
isSdmOptionUseAscii: true
sdmMetaReadPerm:          15
sdmFileReadPerm:          15
sdmReadCounterRetrievalPerm: 2
-----
= FileSettings =
fileNumber: 3
fileType: n/a
commMode: FULL
accessRights RW:          3
accessRights CAR:         0
accessRights R:           2
accessRights W:           3
fileSize: 128
= Secure Dynamic Messaging =
isSdmEnabled: false
isSdmOptionUid: false
isSdmOptionReadCounter: true
isSdmOptionReadCounterLimit: false
isSdmOptionEncryptFileData: false
isSdmOptionUseAscii: true
sdmMetaReadPerm:          14
sdmFileReadPerm:          14
sdmReadCounterRetrievalPerm: 14
=====
content of file 01 length: 32 data: 000f20003a00340406e104010000ff0506e10500808
-----
content of file 02 length: 256 data: 00abd101a7550068747470733a2f2f73646d2e6e66
ASCII Data: ??U??https://sdm.nfcdeveloper.com/tag?picc_data=*****
-----
content of file 03 length: 144 data: 000102030405060708090a0b0c0d0e0f1011121314

```

```
-----  
== FINISHED ==
```

Unset the tag to default settings

This is the “undo” activity within our app: it resets all settings and file content to the default = factory settings. Especially when working with diversified keys some errors during setting may occur. In that case it is helpful to unset the tag to the default settings with this activity before and then run the “Prepare tag” activity again, followed by the setting you want to test.

Tag Personalization

If you are planning to setup an own system based on NTAG 424 DNA tags you should consider the following hints:

- change all application keys (key number 0 to 4) to your own (custom) keys even if you don't use them
- change the file settings for file 02 to an application key (the fabric file settings for the file 01 needs to remain “Eh” meaning free access for the NDEF message)
- setup the tag for **Random Tag UID** (this is a one time setting and you cannot reverse this setting)
- don't use too long NDEF messages as some NFC readers have a limited buffer to read the message

Summary

The Secure Dynamic Message feature is a fascinating way of providing secure information using the standardized way of a NDEF message. This is working on all smartphones having a browser and internet access without installing an app on the user's smartphone. I showed you how to use the NTAG 424 DNA library to get the desired result.

That's it for this tutorial, you find the **source code for the complete app** in my GitHub repository <https://github.com/AndroidCrypto/Ntag424SdmFeature>.

If you want to use a **pre compiled app** I'm providing a “**ready to use app**”, compiled in debug mode in the subfolder “sampleapp”.

Happy coding !